

AgriAgent: Autonomous Dual-Brain Multi-Agent Vision Framework for Zero-Shot Agricultural Image Analysis & Selective Spray Mapping

GitHub Repository: <https://github.com/Chaitanya-666/agriAgent>

Project Guide & Supervisor:

Prof. V. D. Dhore

Associate Professor, Dept. of Computer Engineering

Project Team (Final Year B.Tech):

Chaitanya Shinde (Reg: 231070066)

Amit Ingle · Sahil Chavan · Sumant Vetal

Executive Abstract (Layman & High-Level Summary)

The Everyday Problem: When farmers manage fields, invasive weeds and pest lesions compete with crops for nutrients and water. Today, the only widely available solution is **blanket spraying**—dousing the entire field with expensive chemical herbicides and pesticides. This poisons fertile soil, contaminates groundwater, damages healthy crops, and costs farmers enormous sums of money. While modern artificial intelligence (AI) can detect weeds, existing AI models are rigid: a model trained in Europe on sugar beets completely fails when brought to Maharashtra to inspect Indian cotton fields, because the weeds, leaves, and soils look completely different. Retraining AI for every new crop requires months of work, thousands of hand-drawn image labels, and expensive computing clusters.

Our Solution — AgriAgent: AgriAgent is an autonomous, zero-shot computer vision system that requires **zero retraining**. A farmer or agronomist uploads a single image from a smartphone or drone and types a plain English command, such as: “Find all weeds and spare the cotton plants” or “Locate yellow leaf disease spots”.

AgriAgent instantly combines open-vocabulary detection (**Grounding DINO**) with promptable pixel segmentation (**SAM 2**). It identifies every weed with millimeter accuracy, automatically runs an error-correction loop, and generates a precision spray map. Instead of spraying 100% of the field, the farmer sprays only the 20% to 30% where weeds actually exist—**saving over 70% in chemical costs immediately**.

The Architectural Novelty: Beyond computer vision, AgriAgent introduces a **Dual-Brain Agent Architecture**. Traditional AI agents use heavy, slow Large Language Models (LLMs) for every micro-decision, causing 8–10 second delays and hallucination risks. AgriAgent splits the brain: a sub-50ms non-generative **System-1 decision engine** (inspired by Jev and open-source Laya) handles instant routing, safety boundaries, and refinement checks, while a deliberative **System-2 engine** (Groq LLM + ICAR Agronomy RAG) provides agricultural advice and dosage prescriptions. Built on top of the established **SmartDesk** multi-agent infrastructure, AgriAgent provides a unified, production-grade framework across 5 distinct agricultural applications.

1. Problem Statement & Research Motivation

Conventional weed and crop disease management in agriculture relies on manual scouting by field laborers or indiscriminate broadcast spraying across entire acreages. According to precision agriculture studies by Iowa State University and PhenoRob, broadcast spraying results in severe chemical wastage: up to **76% of applied herbicide falls on bare soil or healthy crops** rather than target weeds. This accelerates herbicide-resistant weed biotypes, degrades soil microbiomes, and imposes heavy financial burdens on smallholder farmers in regions like Maharashtra and Punjab.

While site-specific weed management (SSWM) via smart sprayers offers a compelling remedy, widespread deployment is hindered by the **domain generalization bottleneck** of existing computer vision models:

1. **Supervised CNN Fragility:** Classical architectures (YOLOv8, U-Net++, DeepLabV3+) achieve high precision only on the exact distributions they were trained on. A model trained on German sugar beet fields (Bonn benchmark) experiences severe domain collapse when deployed to Indian cotton (*Gossypium hirsutum*) fields due to divergent foliage geometry, canopy closure, and regional weed biotypes (e.g., *Parthenium*, *Amaranthus*).
2. **Annotation Cost Prohibitive:** Generating pixel-level polygonal ground-truth masks for every regional crop-weed permutation costs hundreds of human hours and thousands of dollars, making supervised retraining infeasible for localized agriculture.
3. **Agent Latency & Reliability Gap:** Emerging multi-agent AI frameworks (LangGraph, AutoGen) attempt agricultural orchestration by piping full text prompts and image contexts into heavy frontier LLMs. This incurs prohibitive latency (> 8 seconds per decision), exorbitant API token consumption, and catastrophic context window overflows when raw segmentation masks are serialized into chat histories.

Formal Objective: To build **AgriAgent**, a zero-shot, open-vocabulary multi-agent framework that accepts arbitrary natural language prompts to detect, segment, and evaluate field crop-weed imagery without domain-specific retraining, producing actionable spray coordinates and agronomic prescriptions under strict sub-2.5 second latency constraints.

2. Dual-Brain System Architecture & Technological Innovation

AgriAgent formulates agricultural image understanding through a biologically inspired **Dual-Brain Cognitive Architecture** that decouples fast, deterministic reactive decisions from slow, deliberative agronomic reasoning.

2.1 System 1: Fast Deterministic Decision & Triage Engine (Jev / Laya Principles)

Modern agent frameworks suffer because they utilize generative LLMs for tasks that require classification and deterministic gating. AgriAgent integrates a dedicated **System-1 non-generative decision layer**:

- **Underlying Principle:** Inspired by newly pioneered “System One” decision models like **Jev** (TypeSafe AI) and its leading open-source counterpart **Laya** (Apache 2.0, ConVAI Innovations), System 1 processes input states in a single parallel pass without token-by-token autoregression.
- **Micro-Second State Routing:** Given a user prompt, System 1 maps the intent to a strictly typed schema (`TaskRoute.WEED_SPRAYING`, `TaskRoute.PEST_AUDITING`, `TaskRoute.CANOPY_COVERAGE`) in < 40ms without invoking an external LLM.
- **Refinement Quality Gating:** Instantly evaluates the predicted Intersection-over-Union (IoU_{pred}) from SAM 2. If $IoU_{pred} < 0.85$, it autonomously triggers the corrective centroid generator without human or LLM intervention.

- **Safety Guardrails:** Enforces strict agricultural physical boundaries (e.g., verifying that spray buffers do not exceed 15cm and herbicide volume remains within Central Insecticide Board limits).

2.2 System 2: Deliberative Agronomic Intelligence & Multimodal RAG

When deep agricultural reasoning or contextual advice is required, the Orchestrator dispatches tasks to **System 2**:

- **Reasoning Core:** Powered by Groq-accelerated Llama-3.3-70B and multimodal vision models (e.g., Qwen2.5-VL / Florence-2).
- **ICAR & CIBRC Agronomy RAG:** Connected to a local ChromaDB vector store indexed with official compendiums from the Indian Council of Agricultural Research (ICAR) and Central Insecticide Board & Registration Committee (CIBRC).
- **Actionable Output:** Rather than just outputting raw labels, System 2 generates a farmer-facing prescription: “Targeted spray map generated for Parthenium hysterophorus. Recommended chemical: Glyphosate 41% SL spot application. Estimated herbicide reduction: 74.2%.”

2.3 Zero-Shot Vision Foundation Pipeline

The vision subsystem consists of two state-of-the-art vision transformer foundation models operating in cascade:

1. **Grounding DINO-T (Open-Vocabulary Object Detector):** Uses a Swin Transformer visual backbone and BERT language backbone with a bidirectional Feature Enhancer and Language-Guided Query Selection module. It aligns text noun phrases (e.g., "weed", "cotton leaf") directly to bounding boxes $[x_1, y_1, x_2, y_2]$ without prior training on agricultural datasets.
2. **SAM 2-Tiny (Segment Anything Model 2):** Uses a hierarchical Hiera vision transformer to ingest bounding boxes as prompts and output dense, pixel-accurate binary masks ($M \in \{0, 1\}^{H \times W}$) and predicted IoU scores in under 125ms on a standard T4 GPU.
3. **Deterministic Heuristic Refinement Loop:** If predicted IoU falls below 0.85, the pipeline computes the spatial centroid of under- or over-segmented regions between the box and mask boundary, injecting corrective positive/negative point prompts back into SAM 2 (capped at 2 iterations to avoid infinite loops).

3. SmartDesk Heritage & Architectural Specialization

AgriAgent directly inherits and expands upon the architectural innovations pioneered in **SmartDesk**, a multi-agent framework previously engineered by team members:

- **What We Pruned:** SmartDesk’s generic desktop productivity tools (Gmail SMTP connectors, Google Calendar synchronization, Telegram bots, local bash shell execution) were excised as they represent bloat for field deployability.
- **What We Preserved & Specialized:**
 - ▶ **The Supervisor-Worker Orchestrator:** The core LangGraph state machine governing task planning, execution dispatch, and completion conditions.
 - ▶ **Clean Slate Finalizers (RemoveMessage):** At the conclusion of every vision task, intermediate scratchpad messages and tool tokens are scrubbed from the state, preventing cross-image context contamination during batch field processing.
 - ▶ **The Structured Artifact Subsystem:** Large 2D binary numpy arrays and GeoTIFF spray rasters are never dumped into chat history. They are registered as typed artifacts (ArtifactType.VISION_MASK, ArtifactType.SPRAY_MAP), preserving LLM context token health.
 - ▶ **ChromaDB Knowledge Engine:** Shifted from general document search to specialized agronomic literature retrieval.

4. Mathematical Formulation & Metrics

The framework's performance and real-world efficacy are measured using rigorous mathematical formulations:

4.1 Segmentation Quality Metrics

For predicted binary mask P and ground-truth mask G :

$$\text{IoU}(P, G) = \frac{|P \cap G|}{|P \cup G|}, \quad \text{Dice}(P, G) = \frac{2 |P \cap G|}{|P| + |G|}$$

4.2 Error Centroid Heuristic Formulation (Refinement Loop)

Let B be the binary bounding box mask and M be the predicted mask. Under-segmentation region $E_u = B \wedge \neg M$ and over-segmentation region $E_o = M \wedge \neg B$. The corrective point (x_c, y_c) and polarity $p \in \{+1, -1\}$ are computed as:

$$(x_c, y_c) = \begin{cases} \left(\frac{1}{|E_u|} \sum_{(x,y) \in E_u} x, \frac{1}{|E_u|} \sum_{(x,y) \in E_u} y \right) & \text{with } p = +1 \text{ if } |E_u| \geq |E_o| \\ \left(\frac{1}{|E_o|} \sum_{(x,y) \in E_o} x, \frac{1}{|E_o|} \sum_{(x,y) \in E_o} y \right) & \text{with } p = -1 \text{ if } |E_o| > |E_u| \end{cases}$$

4.3 Herbicide Savings Percentage Formulation

Let I be the total image pixel area. Let M_w be the weed mask, and let K_r be a circular morphological dilation structuring element representing a spray nozzle safety buffer radius ($r = 5\text{cm}$ to 10cm):

$$M_{\text{spray}} = M_w \circ K_r, \quad \text{Herbicide Savings \%} = \left(1 - \frac{|M_{\text{spray}}|}{|I|} \right) \cdot 100$$

5. Target Datasets & Benchmarking Strategy

To prove true zero-shot cross-domain generalization, the project evaluates models across both international benchmarks and Indian regional staples without fine-tuning:

1. **Sugar Beets 2016 (University of Bonn / PhenoRob)**: Global gold standard dataset consisting of 25,429 multi-spectral UGV images with pixel-wise semantic annotations of crop vs weed (21.57 GB full; 485 MB benchmark sample).
2. **CottonWeeds (Kaggle Indian Benchmark)**: 7,578 high-resolution RGB images of weeds and cotton fields typical to the Indian subcontinent.
3. **Cotton-Weed-12-Class (CottonWeedDet12)**: 5,648 images with 9,000+ bounding box annotations spanning 12 invasive weed species in cotton cropping systems.
4. **Cotton Plant Disease & Pest Dataset**: 2,000+ field images classifying healthy cotton against Aphids, Armyworms, Bacterial Blight, and Leaf Curl Virus.
5. **Global Wheat Head Detection (GWHD 2021)**: 6,515 images from 12 countries used to validate Application 3 (yield counting).

6. Equal 4-Way Team Responsibilities & Work Allocation Matrix

To ensure absolute academic fairness, balanced intellectual rigor, and complete defensibility during the university viva voce, the project is structured into four distinct, equally weighted engineering tracks. Every track involves substantial machine learning, research, or advanced systems design.

Policy for Role Finalization

Status: Track 1 is locked by project author Chaitanya Shinde. Tracks 2, 3, and 4 represent balanced 25% workloads open for immediate review and assignment among Amit Ingle, Sahil Chavan, and Sumant Vetal. If no grievances or counter-proposals are raised by tonight, these four tracks stand as the official team division.

Track & Assignee	Core Technical Components	Key Deliverables & Research Scope	Viva Defense Focus
Track 1: Core AI & Vision Systems LOCKED: Chaitanya Shinde	• Grounding DINO-T + SAM 2 pipeline • System-1 Fast Decision Engine (Laya/Jev) • Deterministic IoU Refinement Engine • LangGraph State Machine & SmartDesk refactor	• Python wrappers (grounding_dino.py, sam2_wrapper.py) • Prompt engineering across 5 agri-domains • Sub-50ms reactive routing & safety guardrails • T4 GPU FP16 memory optimization	Zero-shot foundation models, open-set detection math, and heuristic refinement loop design.
Track 2: Multimodal VLM & Agronomy Intelligence Candidate: Amit Ingle	• Vision-Language Models (Qwen2.5-VL / Florence-2) • Multimodal visual reasoning critic • ICAR & CIBRC Agronomy RAG pipeline • Treatment & chemical dosage generator	• ChromaDB vector store indexing ICAR cotton guides • Zero-shot pest lesion severity diagnosis • Contextual farmer natural-language prescription • VLM vs Heuristic refinement ablation study	Multimodal VLM architectures, agronomic RAG retrieval fidelity, and chemical safety reasoning.
Track 3: Data Science & Empirical Benchmarking Candidate: Sahil Chavan	• Dataset acquisition (Kaggle Cotton, SugarBeets) • Quantitative evaluation harness (iou_dice.py) • Statistical hypothesis testing (bootstrapping) • Herbicide volume reduction mathematical proofs	• 100-image verified cotton benchmark test set • Ground-truth polygon to mask conversion • mIoU, Dice, mAP at 0.5, and latency benchmark tables • Bootstrapped 95% confidence intervals ($p < 0.05$)	Statistical validation, cross-domain transfer metrics, and experimental rigor.
Track 4: Geospatial Systems & Full-Stack Deployment Candidate: Sumant Vetal	• Production Streamlit Dashboard (app/app.py) • Real-time OpenCV / Supervision mask overlay • Geospatial spray-map generator (GeoJSON / GPS) • Automated PDF prescription report compiler	• Interactive live demo portal with 5-app selector • Dual-panel before/after mask visualizer • Drone/tractor spray route coordinate plotter • Farm chemical inventory & savings calculator	Full-stack system integration, geospatial raster rendering, and real-time UI/UX throughput.

6.1 Hardware Feasibility & Local Environment Requirements by Role

A major failure mode in undergraduate engineering projects is unverified dependency on institutional lab servers (e.g., college SSH access). In practice, campus servers frequently suffer from strict firewall port restrictions, unannounced maintenance reboots, outdated CUDA toolchains (incompatible with PyTorch 2.4+ and SAM 2), and resource contention among multiple student batches.

To eliminate this vulnerability, **AgriAgent is architected to be 100% self-sufficient on basic student laptops and zero-cost cloud tiers**. Not a single team member requires a high-end gaming laptop or local GPU to complete their deliverables:

Track & Assignee	Minimum Local Machine Specs	Free Cloud / Engine Tier	Cost & Lab Dependency
Track 1: Core AI & Vision Chaitanya Shinde	- Standard Laptop (8–16 GB RAM) - Core i3/i5 or Ryzen 3/5 CPU - No local GPU required (CPU fallback)	- Google Colab Free Tier (T4 GPU, 16 GB VRAM) - Combined FP16 footprint is only $\approx 430\text{MB}$ - Gives $37\times$ safety headroom on free T4	Zero Cost (Free) Zero Lab Dependency
Track 2: Multi-modal VLM & RAG Candidate: Amit Ingle	- Standard Laptop (8 GB RAM) < 1GB free disk space - ChromaDB vector store runs 100% on CPU	- Groq API Free Tier (30 req/min, 0% local GPU) - HuggingFace Inference API for embeddings - Google Colab T4 for optional local VLM tests	Zero Cost (Free) Zero Lab Dependency
Track 3: Data & Benchmarking Candidate: Sahil Chavan	- Standard Laptop (8 GB RAM) 5–10 GB disk for Kaggle dataset splits - IoU & Dice scripts run in < 1ms on CPU	- Kaggle Free Notebooks (30 hrs/wk Dual T4 GPUs) - Google Drive / Kaggle API for direct dataset pulls - NumPy bootstrapping runs instantly on CPU	Zero Cost (Free) Zero Lab Dependency
Track 4: Geospatial & UI Candidate: Sumant Vetal	- Any basic laptop (Windows / Mac / Linux) 8 GB RAM, standard integrated graphics - OpenCV and Streamlit run natively on CPU	- Localhost development (streamlit run app.py) - Streamlit Community Cloud (free 1-click deploy) - GeoJSON and GPS raster rendering in Python	Zero Cost (Free) Zero Lab Dependency

7. Compressed 4-Week Sprint-to-Finish Roadmap (October 2026 Deadline)

Accelerated Execution Strategy (October 1 – October 31, 2026)

To target full project completion and defense readiness by the end of October 2026, the 16-week timeline is compressed into an intensive **4-Week (30-Day) Concurrent Engineering Sprint**. Rather than relying on serial dependencies, all four tracks execute in parallel against strictly defined data and interface contracts.

Week & Date Window	Core Engineering Sprints & Deliverables	Primary Leads
Week 1 (Oct 1 – Oct 7)	• Foundation & State Schema: Patch <code>src/state.py</code> with typed artifacts and task models. • Standalone Colab Verification: Run Grounding DINO-T + SAM 2-Tiny on 5 sample cotton images. • Data Curation: Download and stage 50 clean Indian Cotton and SugarBeet test images. • UI Shell: Construct Streamlit layout (<code>app/app.py</code>) with upload widget and mock visualizer. Milestone 1 (Tuesday Oct 6/7): Working prototype progress demo for Prof. V. D. Dhore.	All 4 Members (Chaitanya, Amit, Sahil, Sumant)
Week 2 (Oct 8 – Oct 15)	• Model Wrappers: Deploy FP16 inference scripts for Grounding DINO and SAM 2. • LangGraph Assembly: Wire Orchestrator state machine with Clean Slate finalizers. • System-1 Fast Routing: Implement sub-40ms intent classification and safety boundary checks. • Agronomy Knowledge Base: Ingest essential ICAR cotton protection guidelines into ChromaDB. Milestone 2 (Oct 15): Automated CLI pipeline running zero-shot prompt to mask generation.	Chaitanya (AI/State) Amit (VLM/RAG)
Week 3 (Oct 16 – Oct 22)	• Refinement Engine: Implement IoU error centroid calculation and corrective point injection. • Quantitative Benchmark: Evaluate 50-100 images; compute mIoU, Dice, and Herbicide savings %. • Full-Stack Integration: Connect Streamlit dashboard to vision engine with real-time overlay. • Geospatial Engine: Generate GPS coordinates, GeoJSON files, and drone spray route plots. Milestone 3 (Oct 22): Full-stack interactive web application running live with 70%+ savings proof.	Chaitanya (Refine) Sahil (Benchmarking) Sumant (UI & Maps)
Week 4 (Oct 23 – Oct 31)	• Statistical Proofs: Compute bootstrapped 95% confidence intervals and paired t-test tables. • Thesis Monograph (Black Book): Compile comprehensive final B.Tech documentation. • Demo Asset Freeze: Record high-definition end-to-end video walkthrough and prepare viva slides. • Repository Freeze: Final code review, tagging release v1.0.0, and viva defense rehearsal. Milestone 4 (Oct 31): Final B.Tech project submission, Black Book delivery, and viva defense ready.	Sahil (Stats/Tables) All Members (Thesis & Viva Deck)

This document constitutes the official architectural baseline and operational project specification for AgriAgent.

Submitted for review to Project Supervisor Prof. V. D. Dhore, VJTI Mumbai.